

SIMATS ENGINEERING ASSESSMENT TOOL 1

CSA1223

COURSE NAME:

COMPUTER ARCHITECTURE

COURSE CODE:

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)

Assessment Tool Weightage: 40%

68%
48

QUESTIONS:5

Total Marks:25-100

ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

I _____ VARSHA VARDHINI S (192511488)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: §. Varsha

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch-decode-execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly.

However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes

of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

ANSWERS

1. Smart City Surveillance System

Analysis of the Problem

In the given smart city surveillance system, multiple surveillance cameras continuously generate high-resolution video streams. These cameras act as **I/O devices** and continuously transfer data to the **main memory (RAM)**. The **CPU** fetches the video frames from memory, decodes the instructions, executes image-processing algorithms, and identifies suspicious activities. During peak hours, the number of incoming video streams increases significantly. As a result, the CPU, memory, and I/O devices simultaneously request access to the system bus, leading to congestion, increased memory access time, CPU waiting time, and delayed threat detection.

Interaction between CPU, Memory and I/O

I/O devices (Cameras): Capture continuous video frames and send them to the system.

Memory (RAM): Temporarily stores the incoming video frames and program instructions.

CPU: Fetches instructions and video data from memory, processes the frames, and generates the final threat detection output.

When video traffic increases, the CPU cannot process all the data immediately because it spends additional clock cycles waiting for memory access. This reduces overall system performance and increases response time.

Impact of Bus Structure

In a **single-bus architecture**, the CPU, memory, and I/O devices share one common

communication bus. Since only one transfer can occur at a time, simultaneous requests from multiple cameras create **bus contention**, resulting in delays and lower throughput.

In contrast, a **multi-bus architecture** provides separate buses for CPU-memory communication and I/O communication. While one bus transfers video frames from cameras to memory, another bus allows the CPU to fetch instructions and process data simultaneously. This parallel communication minimizes waiting time, increases bandwidth, and significantly improves the efficiency of the surveillance system.

Improving the Instruction Execution Cycle

The CPU follows the instruction execution cycle:

Fetch → Decode → Execute → Store

Performance can be enhanced by:

Instruction Pipelining: Executes multiple instructions simultaneously, reducing execution time.

Cache Memory: Stores frequently accessed instructions and video data, reducing RAM access delays.

Direct Memory Access (DMA): Enables video data to be transferred directly from I/O devices to memory without continuous CPU involvement.

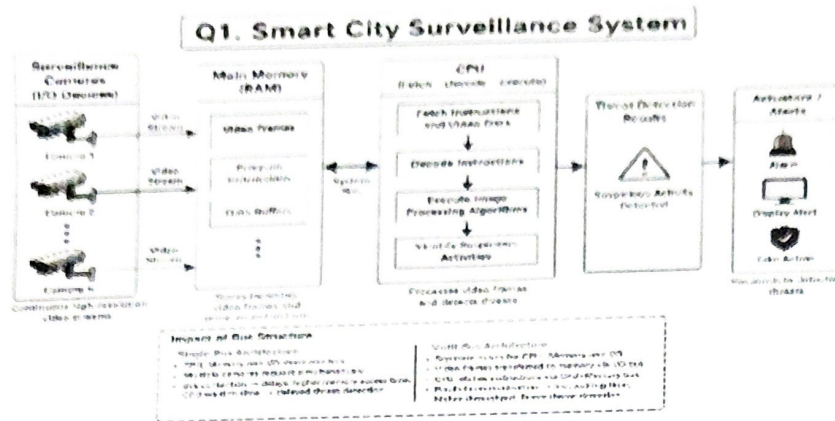
Branch Prediction: Reduces delays caused by conditional instructions.

Parallel Processing: Allows simultaneous processing of multiple video streams.

These improvements reduce CPU idle time, improve throughput, and ensure faster threat detection even during peak traffic.

Conclusion

Therefore, the surveillance system can achieve faster and more reliable real-time threat detection by adopting a **multi-bus architecture**, optimizing the **instruction execution cycle**, implementing **cache memory**, **DMA**, and **instruction pipelining**. These architectural improvements minimize bus contention, reduce CPU waiting time, and significantly enhance overall system performance.



2. Real-Time Stock Trading Application

Analysis

A real-time stock trading system executes millions of instructions every second. During high load, **frequent memory access** and **branch instructions** increase the **Cycles Per Instruction (CPI)**, causing delays in transaction processing.

Impact of Fetch–Decode–Execute Cycle

Fetch: CPU fetches instructions from memory. Frequent memory access increases fetch time.

Decode: Branch instructions require additional decoding and may cause pipeline stalls.

Execute: The CPU waits if required data is unavailable, increasing execution time.

Write Back: Results are stored in registers/memory, and heavy memory traffic causes additional delay.

High CPI → More Clock Cycles → Increased Execution Time → Delayed Transactions

Methods to Reduce CPI

Cache Memory: Reduces memory access time.

Instruction Pipelining: Executes multiple instructions simultaneously.

Branch Prediction: Reduces delays caused by branch instructions.

Out-of-Order Execution: Executes independent instructions while waiting for memory.

Superscalar Architecture: Executes multiple instructions in one clock cycle.

High-Speed Memory: Improves data access speed.

Architectural Enhancements

Multi-core processors

L1/L2/L3 Cache

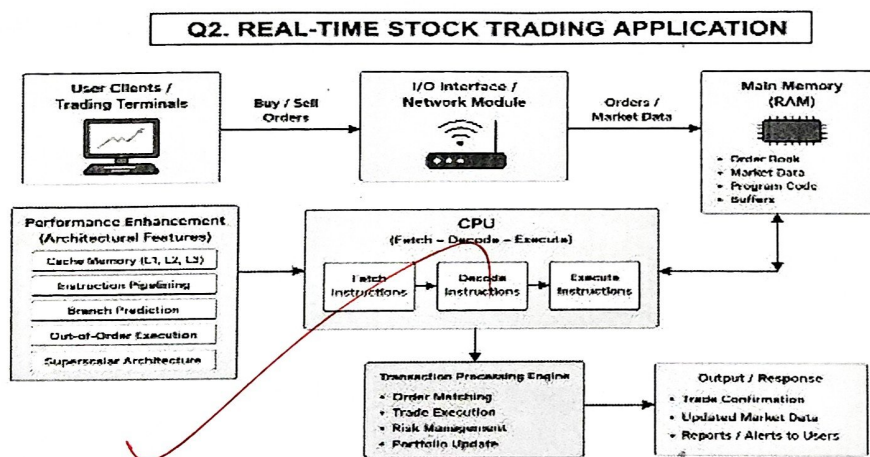
Pipeline optimization

Branch prediction

Faster memory hierarchy

Conclusion

Using cache memory, pipelining, branch prediction, superscalar execution, and faster memory systems reduces CPI, improves CPU performance, decreases execution time, and enables faster real-time transaction processing.



3. 8085 Temperature Control System

Analysis

In an industrial temperature control system, sensors continuously send temperature data to the **8085 microprocessor**, which processes the data and controls actuators such as cooling fans or heaters. If the program uses an **incorrect addressing mode**, the CPU may read data from the wrong register or memory location,

resulting in incorrect temperature readings and improper control actions.

Influence of Addressing Modes on Data Access

Immediate Addressing: Data is provided directly in the instruction (e.g., MVI A,25H). Suitable for fixed values.

Register Addressing: Data is transferred between registers (e.g., MOV A,B). Faster because data is inside the CPU.

Direct Addressing: Data is accessed from a specified memory location (e.g., LDA 2050H). Used for stored sensor values.

Register Indirect Addressing: The memory address is stored in the HL register pair (e.g., MOV A,M). Useful for sequential sensor data.

Possible Causes of Errors

Incorrect memory address used.

Wrong addressing mode selected.

Sensor data overwritten in memory.

Improper register initialization.

Invalid instruction sequence leading to incorrect data access.

Effective Use of 8085 Instruction Set

LDA / STA – Read and store sensor values correctly.

MOV / MVI – Transfer data between registers.

LXI – Load correct memory addresses.

CMP – Compare temperature with the reference value.

JZ / JC / JNC – Make correct control decisions.

HLT – Stop execution after successful operation.

Conclusion

Stack Segment (SS): Stores return addresses, local variables, and temporary data using the **Stack Pointer (SP)**

Causes of Incorrect Data Access

Incorrect initialization of segment registers (CS, DS, SS).

Wrong offset address.

Accessing data from the wrong segment.

Incorrect addressing mode selection.

Causes of Stack Overflow

Excessive **PUSH** operations without corresponding **POP** operations.

Deep function calls or recursion.

Improper Stack Pointer (SP) management.

Managing Instruction Execution

Execute instructions using the correct **CS:IP** combination.

Access variables through the correct **DS**.

Use **SS:SP** properly for stack operations.

Maintain the sequence **Fetch → Decode → Execute → Write Back** to ensure correct execution.

9

Managing Addressing Modes

Use **Immediate Addressing** for constants.

Use **Register Addressing** for faster operations.

Use **Direct Addressing** for fixed memory locations.

Use **Based/Indexed Addressing** for arrays and multiple program data.

Clarity

10

Clear, well²
organized, and
properly explained
solution

Understandable¹⁻⁵
with minor clarity
issues

Limited clarity¹
and
explanation

Poorly presented^D
and difficult to
understand